DBMS

PROJECT REPORT

on

StudyLab

Submitted by

(Group No. 13)

Aakriti 240354
Aaryaman Pratap Singh 240356
Abhijeet Gupta 240360
Arnav Gupta 240399

CSE 1 - Batch 2024-2028

Submitted to

Mrs Nishtha Phutela

(School of Engineering and Technology)

BMU
BML Munjal University

CANDIDATE'S DECLARATION

We hereby certify that the work on the project entitled "StudyLab", in partial fulfillment of requirements for the award of the Degree of Bachelor of Technology in the School of Engineering and Technology at BML Munjal University, is an authentic record of our own work carried out during the period from Feb 2026 to April 2026.

Table of Contents

1. Project Overview
2. Problem Definition
3. Proposed Solution
4. ER Diagram
5. Schema Design
6. Keys (Primary, Foreign, Candidate)
7. Normalization (1NF, 2NF, 3NF)
8. Indexing
9. SQL Implementation
10. Output Explanation (Project Queries)
11. Conclusion

1. Project Overview

StudyLabs is a two-client study companion platform built on a single MySQL database. It serves both mentees (students) and mentors (teachers / counsellors / batch leads) through clients tailored to each role, while writing through one normalised relational schema.

Mentee side — Android app

Lets a student organise their study material into colour-coded subjects, drop files / notes / links under each subject, and track behaviour-change habits with daily check-ins and per-question scoring . Each habit is summarised by aggregate statistics (average, variance, trend, AI-generated narrative) and visualised.

Mentor side — Website

Lets a mentor view a roster of assigned mentees, drill into one mentee's habit consistency / vault activity, push bulk tasks and assessments to multiple students at once, and watch a real-time activity-log feed of everything their mentees do.

2. Problem Definition

Modern students rely on multiple platforms such as messaging apps, cloud storage, and video resources to manage their study material. This leads to fragmentation, making it difficult to organize, retrieve, and effectively use academic content. At the same time, students lack structured tools to track important habits like consistent study, revision, and time management, which directly impact their performance.

From the mentor's perspective, there is limited visibility into a student's daily engagement and learning behavior. Performance is often evaluated only after exams, which delays intervention and reduces the effectiveness of guidance. Additionally, there is no unified system to assign tasks, monitor progress, or maintain an activity record.

Thus, the core problem lies in the absence of a centralized, data-driven platform that integrates study management, habit tracking, and mentor oversight, resulting in poor organization, low accountability, and delayed academic support.

Concretely:

For the student	**For the mentor**
Material scattered across many apps.	No centralised mentee roster.
No way to track if a habit is moving the needle.	Cannot tell which mentee is disengaged until exam season.
No accountability — "I'll start tomorrow".	Cannot push the same task to many mentees in one shot.
Daily check-ins live in private notes.	No audit trail of what a mentee did this week.

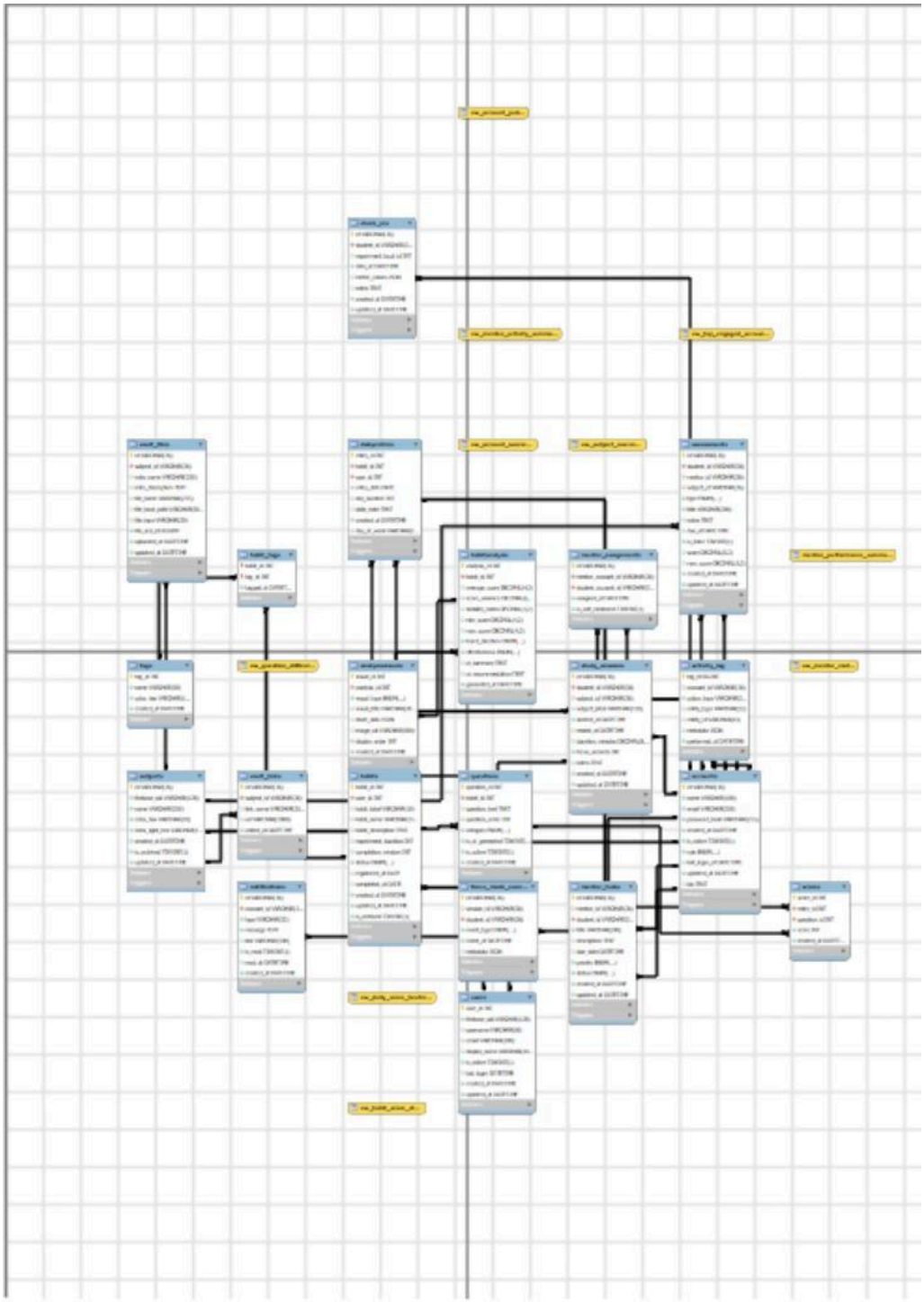
3. Proposed Solution

StudyLab proposes a unified platform built on a centralized MySQL database that integrates study resource management, habit tracking, and mentor monitoring into a single system. It provides two interfaces: an Android application for students and a web dashboard for mentors, both connected through a structured relational schema.

The student interface allows users to organize study material into subjects, store files and links, and track daily habits through structured check-ins. These inputs are processed to generate insights such as averages, trends, and performance summaries. The mentor interface enables monitoring of multiple students, assigning tasks in bulk, and analyzing engagement through activity logs.

The system's database design ensures efficiency and reliability through normalization, indexing, and transactional operations. By combining structured storage with analytical insights, StudyLab enhances both student productivity and mentor decision-making.

5. ER Diagram



accounts(_id_, name, email{U}, password_hash, role, is_active,
last_login_at, bio, created_at, updated_at)

users(_user_id_, firebase_uid{U}, username, email,
display_name, is_active, last_login, created_at, updated_at)

subjects(_id_, firebase_uid (**FK->accounts.id**), name, color_hex,
color_light_hex, is_archived, created_at, updated_at)

vault_files(_id_, subject_id (**FK->subjects.id)**, note_name,
note_description, file_name, file_local_path,
file_type, file_size_kb, uploaded_at, updated_at)

vault_links(_id_, subject_id (**FK->subjects.id),** link_name,
url, added_at)

habits(_habit_id_, user_id (**FK->users.user_id**), habit_label,
habit_name, habit_description, experiment_duration,
completion_window, status, registered_at, completed_at,
is_archived, created_at, updated_at)

questions(_question_id_, habit_id (**FK->habits.habit_id**),
question_text, question_order, category,
is_ai_generated, is_active, created_at)

dailyentries(_entry_id_, habit_id (**FK->habits.habit_id**),
user_id **(FK->users.user_id**), entry_date, day_number,
daily_note, day_of_week (GENERATED), created_at)

scores(_score_id_, entry_id **(FK->dailyentries.entry_id**),
question_id (**FK->questions.question_id**), score, created_at)

habitanalysis(_analysis_id_, habit_id **(FK->habits.habit_id**),
average_score, score_variance, stability_index,
min_score, max_score, trend_direction,
effectiveness, ai_summary, ai_recommendation,
generated_at)

analysisvisuals(_visual_id_, analysis_id (**FK->habitanalysis.analysis_id**),
visual_type, visual_title, chart_data (JSON),
image_url, display_order, created_at)

notifications(_id_, account_id **(FK->accounts.id**), type, message,
link, is_read, read_at, created_at)

activity_log(_log_id_, account_id (**FK->accounts.id, ON DELETE SET NULL**),
action_type, entity_type, entity_id,
metadata (JSON), performed_at)

tags(_tag_id_, name{U}, color_hex, created_at)

habit_tags(_habit_id_, _tag_id_, tagged_at) **-- composite PK, M:N bridge**

mentor_assignments(_id_, mentor_account_id (**FK->accounts.id**),

```
            student_account_id (FK->accounts.id){U},
            is_self_mentored, assigned_at)            -- self-referential M:N

mentor_tasks(_id_, mentor_id (FK->accounts.id),
            student_id (FK->accounts.id), title, description,
            due_date, priority, status, created_at, updated_at)

assessments(_id_, student_id (FK->accounts.id),
            mentor_id (FK->accounts.id, ON DELETE SET NULL),
            subject_id (FK->subjects.id, ON DELETE SET NULL),
            type, title, notes, due_at, is_done, score,
            max_score, created_at, updated_at)
```

5.2 Relationships and cardinality

From	Relationship	To	Cardinality	Notes
accounts	owns	subjects	1 : N	one student → many subjects
subjects	contains	vault_files	1 : N	ON DELETE CASCADE
subjects	contains	vault_links	1 : N	ON DELETE CASCADE
users	tracks	habits	1 : N	ON DELETE CASCADE
habits	has	questions	1 : N	ON DELETE CASCADE
habits	logs	daily entries	1 : N	one habit → many check-ins
dailyentries × questions	answers	scores	M : N (resolved)	bridge: scores
habits	summarised by	habitanalysis	1 : 1 (logically)	refreshed by proc_recompute_habit_analysis
habitanalysis	has charts	analysisvisuals	1 : N	
habits ↔ tags	tagged-with	tags	M : N	bridge: habit_tags
accounts (mentor) ↔ accounts (student)	mentors	accounts	M : N self-ref	bridge: mentor_assignments; UNIQUE on student_account_id ⇒ one mentor / student
accounts	receives	notifications	1 : N	inbox
accounts	performs	activity_log	1 : N	append-only audit
mentor → student	assigns	mentor_tasks	1 : N	one task row per (mentor, student, task)
mentor / student	linked to	assessments	1 : N	nullable mentor_id ⇒ self-tracked OK

5.3 ER diagram explanation

1. The schema is multi-tenant by accounts.id: every domain row eventually traces back to an account, either directly (vault) or via the legacy users table (habits) by joining on email.

2. Three M:N relationships are explicitly resolved into bridge tables (scores, habit_tags, mentor_assignments) — satisfying the rubric's "handling of M:N using bridge entity" criterion.
3. The mentor↔student link is a self-referential M:N on accounts, with a UNIQUE constraint on student_account_id enforcing the business rule "one mentor per student" while still allowing a mentor to mentor many.
4. Every foreign key carries an explicit ON DELETE rule — CASCADE for child rows, SET NULL for activity_log.account_id and assessments.mentor_id so audit history and self-tracked assessments survive a deletion.

6. Schema Design

6.1 Core tables (CREATE TABLE — DDL)

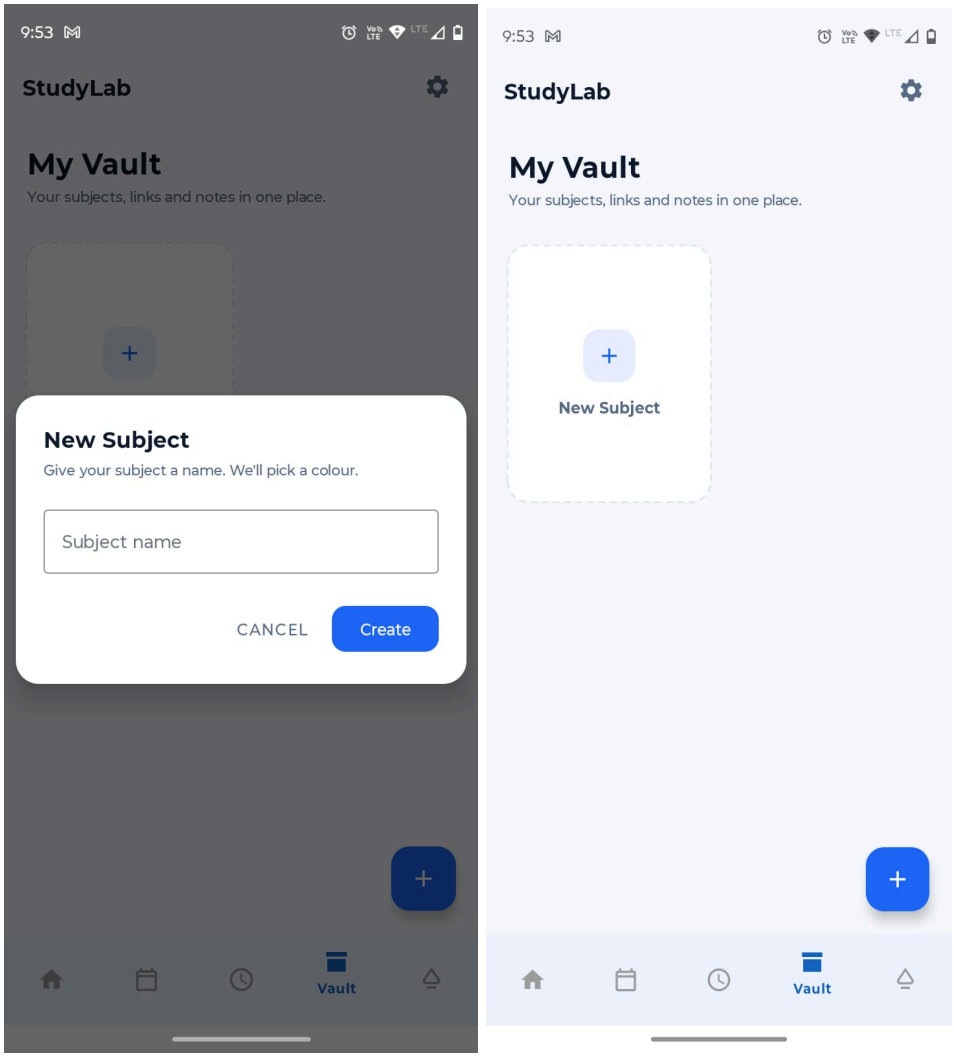
The accounts table is the canonical user record:

```
CREATE TABLE IF NOT EXISTS accounts (
        id          VARCHAR(36)  NOT NULL PRIMARY KEY,
        name        VARCHAR(100) NOT NULL,
        email       VARCHAR(255) NOT NULL UNIQUE,
        password_hash VARCHAR(255) NOT NULL,
        role        ENUM('student','mentor','admin') NOT NULL DEFAULT 'student',
        is_active   TINYINT(1)   NOT NULL DEFAULT 1,
        last_login_at DATETIME      NULL,
        bio         TEXT     NULL,
        created_at          DATETIME          NOT NULL DEFAULT CURRENT_TIMESTAMP,
        updated_at          DATETIME          NOT NULL DEFAULT CURRENT_TIMESTAMP
                            ON UPDATE CURRENT_TIMESTAMP
);
```

The vault branch:

```
CREATE TABLE IF NOT EXISTS subjects (
        id          VARCHAR(36)  NOT NULL PRIMARY KEY,
        firebase_uid        VARCHAR(128) NOT NULL,          -- holds accounts.id
        name                VARCHAR(255) NOT NULL,
        color_hex           VARCHAR(10)  NOT NULL DEFAULT '#22C1A8',
        color_light_hex VARCHAR(10)  NOT NULL DEFAULT '#E1F7F2',
        is_archived         TINYINT(1)   NOT NULL DEFAULT 0,
        created_at          DATETIME          NOT NULL DEFAULT CURRENT_TIMESTAMP,
        updated_at          DATETIME          NOT NULL DEFAULT CURRENT_TIMESTAMP
                            ON UPDATE CURRENT_TIMESTAMP,
        INDEX idx_subjects_uid (firebase_uid)
);

CREATE TABLE IF NOT EXISTS vault_files (
        id          VARCHAR(36)  NOT NULL PRIMARY KEY,
        subject_id          VARCHAR(36)  NOT NULL,
        note_name           VARCHAR(255),
        note_description TEXT,
        file_name           VARCHAR(255),
        file_local_path  VARCHAR(500),
        file_type           VARCHAR(20),
        file_size_kb        BIGINT            NOT NULL DEFAULT 0,
        uploaded_at         DATETIME          NOT NULL DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (subject_id) REFERENCES subjects(id) ON DELETE CASCADE
);
```

Output:



6.2 Special DDL features used

1. GENERATED STORED column on dailyentries.day_of_week — materialised from DAYNAME(entry_date) so calendar-style queries hit an indexed string column.
2. JSON columns on activity_log.metadata and analysisvisuals.chart_data — schemaless audit / chart payloads.
3. ENUM checks on habits.status, accounts.role, mentor_tasks.priority, assessments.type — built-in domain constraints.
4. ON UPDATE CURRENT_TIMESTAMP on every updated_at column — free row-level last-modified tracking with no application code.

7. Keys (Primary, Foreign, Candidate)

Table	Primary Key	Foreign Keys	Candidate Keys
accounts	id (UUID)	—	email (UNIQUE)
users	user_id	—	firebase_uid (UNIQUE), email
subjects	id (UUID)	firebase_uid → accounts.id (loose)	—
vault_files	id (UUID)	subject_id → subjects.id	—
vault_links	id (UUID)	subject_id → subjects.id	—
habits	habit_id	user_id → users.user_id	—
questions	question_id	habit_id → habits.habit_id	(habit_id, question_order)
daily entries	entry_id	habit_id, user_id	(habit_id, user_id, entry_date)
scores	score_id	entry_id, question_id	(entry_id, question_id)
habitanalysis	analysis_id	habit_id	—
analysisvisuals	visual_id	analysis_id	—
notifications	id (UUID)	account_id → accounts.id	—
activity_log	log_id (BIGINT)	account_id → accounts.id (SET NULL)	—
tags	tag_id	—	name (UNIQUE)
habit_tags	(habit_id, tag_id) composite	habit_id, tag_id	composite PK is the only candidate
mentor_assignments	id (UUID)	mentor_account_id, student_account_id	student_account_id (UNIQUE)
mentor_tasks	id (UUID)	mentor_id, student_id	—
assessments	id (UUID)	student_id, mentor_id (SET NULL), subject_id (SET NULL)	—

- **ON DELETE rules — CASCADE** for child rows that should die with the parent (*vault_files with subject, scores with entry, notifications with account, mentor_tasks with either side*);
- **SET NULL** to preserve history (*activity_log.account_id, assessments.mentor_id, assessments.subject_id*).

8. Normalization (1NF, 2NF, 3NF)

8.1 1NF — atomic values

- All scalar columns (name, email, score, entry_date, …) hold a single atomic value — never CSV lists or JSON-encoded business data.
- Repeating groups are extracted into their own tables: a habit's questions live in questions; a daily entry's answers live in scores.
- Every table has a primary key, so each row is uniquely addressable.
- activity_log.metadata and analysisvisuals.chart_data are intentionally schemaless extension points (audit + chart payloads).

8.2 2NF — no partial dependencies on a composite key

- habit_tags(habit_id, tag_id) has only tagged_at, which depends on the full composite key (the moment THIS habit was given THIS tag).

8.3 3NF — no transitive dependencies

The schema actively avoids the canonical 3NF violations:

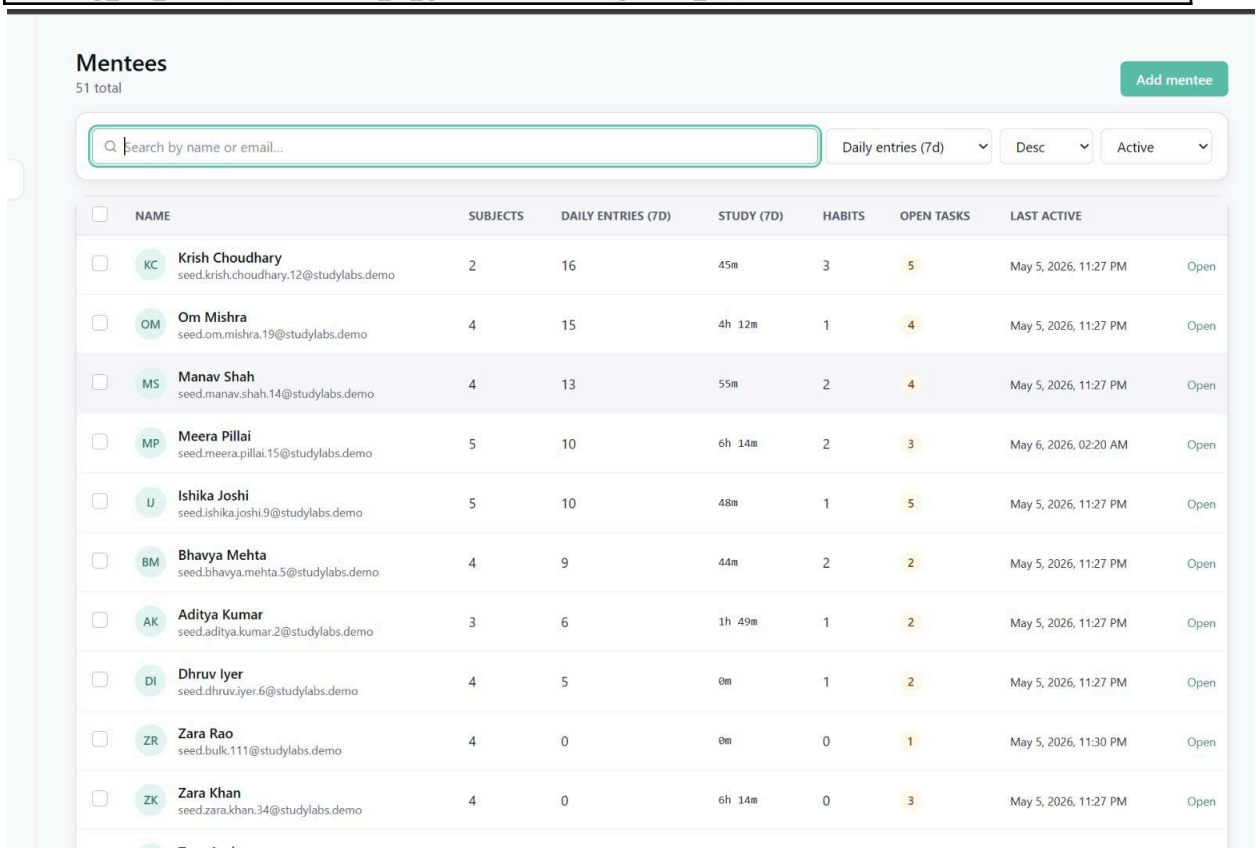
Hypothetical violation	What we did instead
Storing mentor_name on mentor_assignments.	Store only mentor_account_id; join accounts to fetch the name.
Storing subject_name on every vault_files row.	vault_files.subject_id → subjects.id; the name lives on subjects only.
Storing habit_name on every dailyentries row.	dailyentries.habit_id → habits.habit_id; the name lives on habits only.
Storing question text on scores.	scores.question_id → questions.question_id; text lives on questions.
Storing a computed average score on every habit row.	Aggregated on demand by view vw_habit_score_stats or cached in habitanalysis.
Storing day-of-week as a hand-maintained column.	dailyentries.day_of_week is GENERATED STORED from entry_date.
Storing total file count on subjects.	Aggregated by vw_subject_overview; not duplicated.

9. Indexing

9.1 Single-column B-Tree indexes (FK / lookup)

Every JOIN is keyed on a foreign-key column. Without these, every join would force a full table scan on the child.

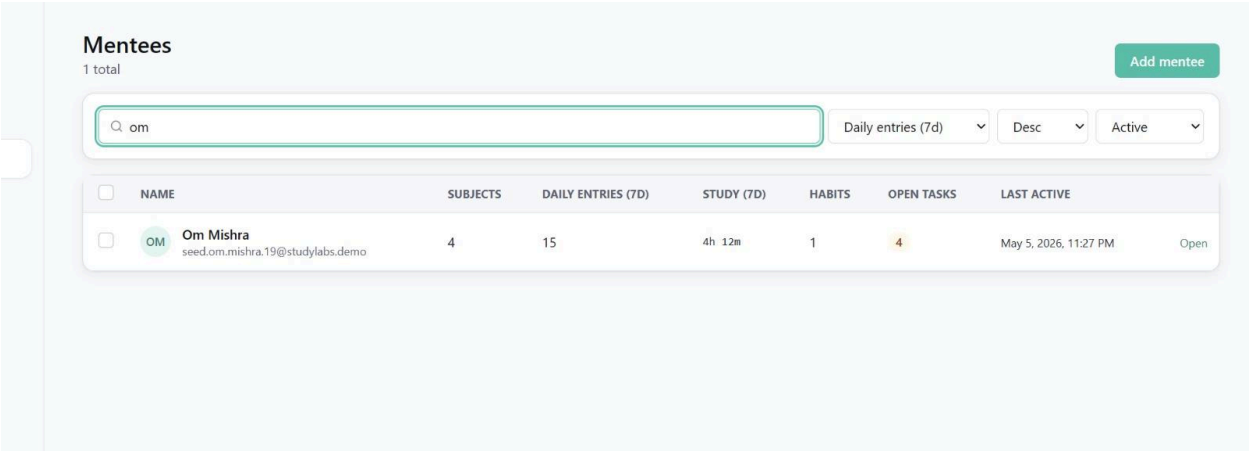
```
CALL sp_add_index('vault_files',  'idx_vf_subject',  'INDEX (subject_id)');
CALL sp_add_index('habits',       'idx_h_user',      'INDEX (user_id)');
CALL sp_add_index('dailyentries', 'idx_de_user',                'INDEX (user_id)');
CALL sp_add_index('dailyentries', 'idx_de_habit',               'INDEX (habit_id)');
CALL sp_add_index('scores',       'idx_sc_entry',   'INDEX (entry_id)');
CALL sp_add_index('scores',       'idx_sc_question', 'INDEX (question_id)');
```



9.2 Composite indexes (filtered ORDER BY)

Most-selective column first (firebase_uid), boolean filter second (is_archived), sort key last (created_at DESC) — the planner serves the entire WHERE … ORDER BY … LIMIT from the index, no filesort.

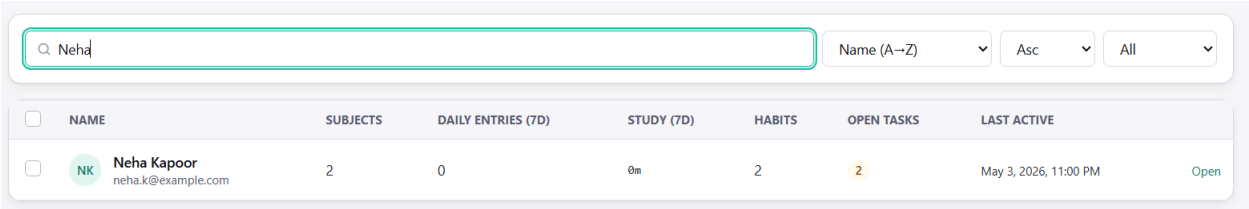
```
CALL sp_add_index('subjects', 'idx_subj_uid_archived',
  'INDEX (firebase_uid, is_archived, created_at DESC)');
CALL sp_add_index('dailyentries', 'idx_de_user_date',
  'INDEX (user_id, entry_date)');
CALL sp_add_index('notifications', 'idx_n_acct_unread',
  'INDEX (account_id, is_read, created_at DESC)');
CALL sp_add_index('activity_log', 'idx_al_acct_when',
  'INDEX (account_id, performed_at DESC)');
```



9.3 FULLTEXT indexes (natural-language search)

Vault search uses MATCH() AGAINST() in natural-language mode for ranked results — LIKE '%foo%' would never use an index.

```
CALL sp_add_index('subjects',    'ftx_subj_name',
  'FULLTEXT INDEX (name)');
CALL sp_add_index('vault_files', 'ftx_vf_text',
  'FULLTEXT INDEX (note_name, note_description, file_name)');
CALL sp_add_index('habits',      'ftx_habit_text',
  'FULLTEXT INDEX (habit_name, habit_description)');
```



10. SQL Implementation

10.1 DDL — Data Definition Language

Already covered in Section 6 and 9 — CREATE TABLE, CREATE INDEX, CREATE VIEW, CREATE TRIGGER, CREATE PROCEDURE / FUNCTION, ALTER TABLE … ADD COLUMN, DROP TABLE.

```
ALTER TABLE reminder_subscriptions
            ADD COLUMN IF NOT EXISTS last_sent_at DATETIME NULL;

DROP TABLE IF EXISTS reminder_subscriptions;
```

10.2 DML — Data Manipulation Language

```
-- INSERT
INSERT INTO tags (name, color_hex) VALUES ('focus', '#22C1A8');

-- UPDATE
UPDATE notifications
   SET is_read = 1, read_at = NOW()
 WHERE id = 'aabb-...';

-- DELETE
DELETE FROM subjects
 WHERE is_archived = 1
   AND created_at  < DATE_SUB(NOW(), INTERVAL 1 YEAR);

-- UPSERT (INSERT ... ON DUPLICATE KEY UPDATE)
INSERT INTO tags (name, color_hex) VALUES ('focus', '#22C1A8')
ON DUPLICATE KEY UPDATE color_hex = VALUES(color_hex);
```

10.3 DQL — Data Query Language (SELECT)

```
SELECT id, name, color_hex, created_at
  FROM subjects
 WHERE firebase_uid = ?      -- :account_id
   AND is_archived  = 0
 ORDER BY created_at DESC
 LIMIT 10;
```

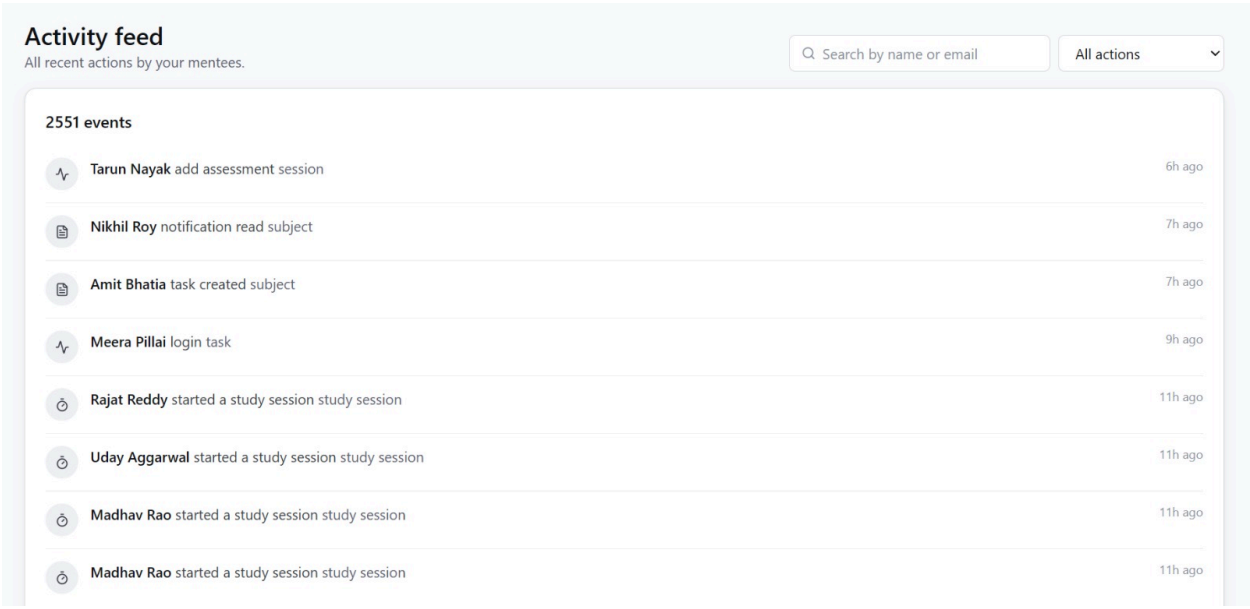
10.4 TCL — Transaction Control (ACID)

The project demonstrates all four ACID properties:

1. **Atomicity** — proc_register_account writes accounts + notifications + activity_log as one unit; an EXIT HANDLER FOR SQLEXCEPTION rolls all three back if any one fails.
2. **Consistency** — foreign keys, ENUM-bounded role/status, the UNIQUE on mentor_assignments.student_account_id, and SIGNAL SQLSTATE '45000' raises keep the DB valid at every boundary.
3. **Isolation** — every multi-write route is wrapped in START TRANSACTION … COMMIT, running at MySQL's default REPEATABLE READ.
4. **Durability** — InnoDB's default settings (innodb_flush_log_at_trx_commit = 1) flush the WAL on every commit.

COMMIT / ROLLBACK demonstrated inside proc_register_account:

```
START TRANSACTION;
        INSERT INTO accounts       (...) VALUES (...);
        INSERT INTO notifications  (...) VALUES (...);
        INSERT INTO activity_log   (...) VALUES (...);
COMMIT;
-- on SQLEXCEPTION: ROLLBACK; RESIGNAL;
```



SAVEPOINT demonstrated inside proc_log_daily_entry:

```
START TRANSACTION;
  INSERT INTO dailyentries (...) VALUES (...);
  SAVEPOINT before_scores;
            -- loop over JSON of {question_id, score}
            -- on a bad score:
            --   ROLLBACK TO SAVEPOINT before_scores;
            --   SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Score must be in [0..10]';
  RELEASE SAVEPOINT before_scores;
COMMIT;
```

The savepoint lets us roll back ONLY the score rows while preserving the parent daily entries row — the mentor still sees that the student attempted to log today even if the question payload was malformed.

10.5 Joins

10.5.1 INNER JOIN — files with their parent subject

```
SELECT vf.id, vf.note_name, vf.file_size_kb,
            s.name     AS subject_name,
            s.color_hex AS subject_color
  FROM vault_files vf
 INNER JOIN subjects s ON s.id = vf.subject_id
 WHERE s.firebase_uid = ?
 ORDER BY vf.uploaded_at DESC;
```

10.5.2 LEFT JOIN — every account, even those with zero subjects

```
SELECT a.id, a.name, a.email,
            COUNT(s.id) AS subject_count
  FROM accounts a
  LEFT JOIN subjects s
            ON s.firebase_uid = a.id
            AND s.is_archived  = 0
 GROUP BY a.id, a.name, a.email
 ORDER BY subject_count DESC;
```

10.5.3 RIGHT JOIN — data-quality check

```
SELECT a.id, a.name, s.id AS subject_id, s.name AS subject_name
  FROM accounts a
 RIGHT JOIN subjects s ON s.firebase_uid = a.id;
```

10.5.4 SELF JOIN — mentor + student names from one table

```
SELECT mentor.name  AS mentor_name,
             student.name AS student_name,
             ma.assigned_at
  FROM mentor_assignments ma
  JOIN accounts mentor  ON mentor.id  = ma.mentor_account_id
  JOIN accounts student ON student.id = ma.student_account_id;
```

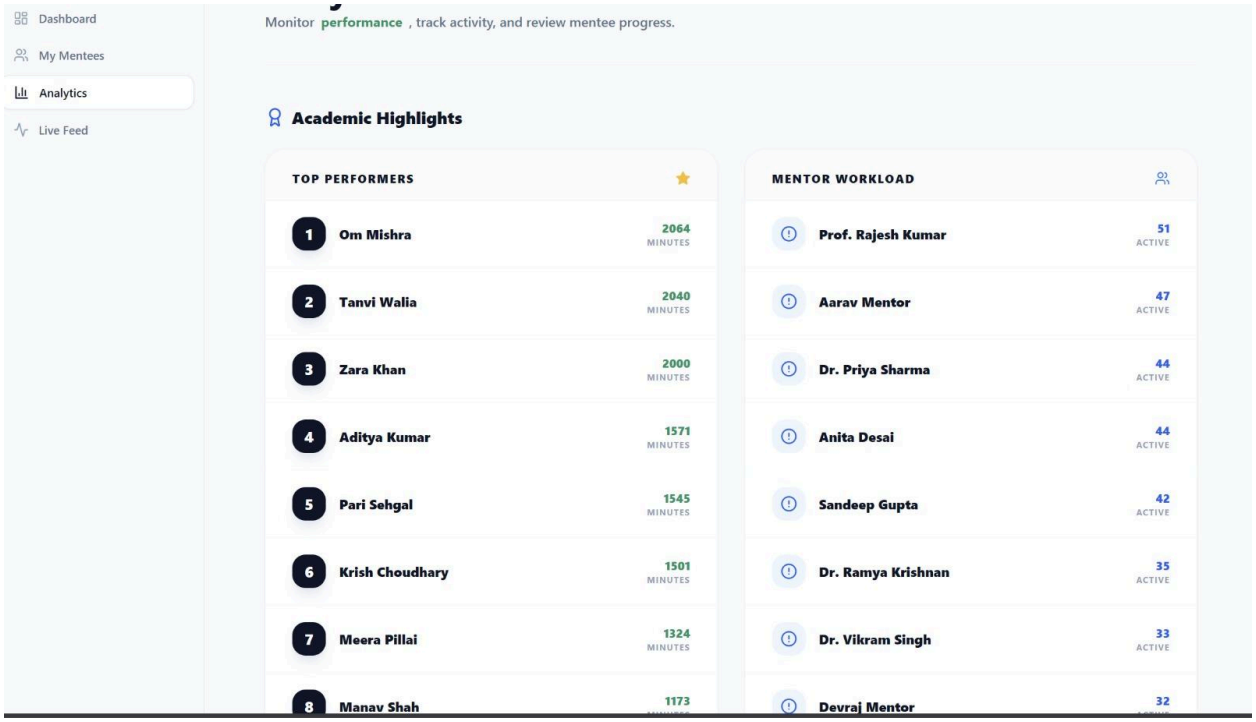
10.5.5 FULL OUTER JOIN — emulated via UNION

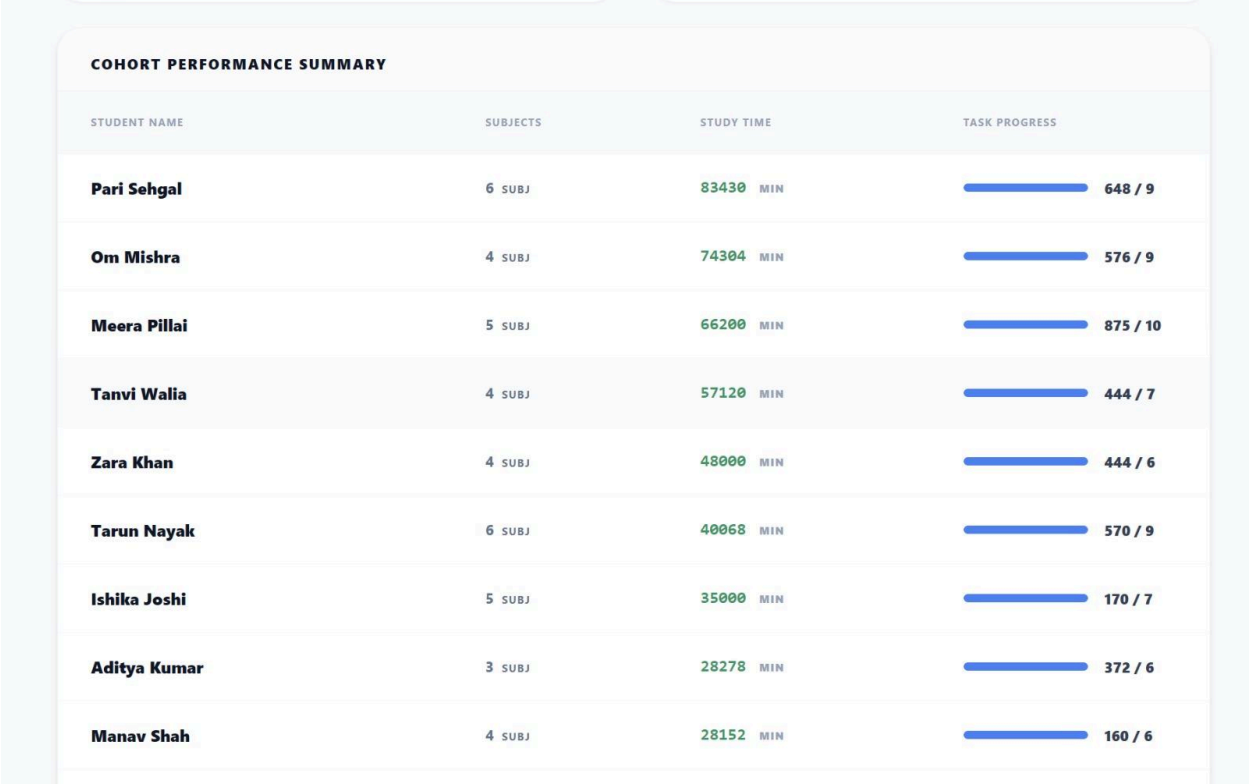
```
SELECT h.habit_id, h.habit_name, ha.analysis_id, ha.average_score
  FROM habits h
  LEFT JOIN habitanalysis ha ON ha.habit_id = h.habit_id
UNION
SELECT h.habit_id, h.habit_name, ha.analysis_id, ha.average_score
  FROM habits h
 RIGHT JOIN habitanalysis ha ON ha.habit_id = h.habit_id;
```

10.6 GROUP BY + HAVING

```
SELECT s.id, s.name,
             COUNT(vf.id)        AS file_count,
             SUM(vf.file_size_kb) AS total_kb
  FROM subjects s
  JOIN vault_files vf ON vf.subject_id = s.id
 GROUP BY s.id, s.name
HAVING COUNT(vf.id) > 1
   AND SUM(vf.file_size_kb) > 100
 ORDER BY total_kb DESC;
```

WHERE filters rows BEFORE aggregation; HAVING filters groups AFTER. Here we ask for subjects with more than one file AND more than 100 KB of storage — neither can be expressed in WHERE because both are aggregates.





COHORT PERFORMANCE SUMMARY

STUDENT NAME	SUBJECTS	STUDY TIME	TASK PROGRESS
Pari Sehgal	6 SUBJ	83430 MIN	648 / 9
Om Mishra	4 SUBJ	74304 MIN	576 / 9
Meera Pillai	5 SUBJ	66200 MIN	875 / 10
Tanvi Walia	4 SUBJ	57120 MIN	444 / 7
Zara Khan	4 SUBJ	48000 MIN	444 / 6
Tarun Nayak	6 SUBJ	40068 MIN	570 / 9
Ishika Joshi	5 SUBJ	35000 MIN	170 / 7
Aditya Kumar	3 SUBJ	28278 MIN	372 / 6
Manav Shah	4 SUBJ	28152 MIN	160 / 6

Explanation : Joins in StudyLab reflect real-world relationships between entities. For example, INNER JOIN is used to fetch files along with their subject details in the vault, while SELF JOIN is used in mentor-student mapping to retrieve both mentor and student names from the same accounts table.

10.7 Subqueries

10.7.1 Non-correlated, in WHERE … IN (…)

```
SELECT vf.id, vf.note_name, vf.subject_id
  FROM vault_files vf
 WHERE vf.subject_id IN (
            SELECT id FROM subjects
            WHERE firebase_uid IN (
            SELECT id FROM accounts WHERE is_active = 1
            )
 );
```

10.7.2 Correlated, with EXISTS

```
SELECT s.id, s.name
  FROM subjects s
 WHERE EXISTS (
            SELECT 1 FROM vault_files vf
            WHERE vf.subject_id  = s.id
            AND vf.uploaded_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
 );
```

10.7.3 Scalar subquery in SELECT

```
SELECT s.id, s.name,
            (SELECT COUNT(*) FROM vault_files vf
            WHERE vf.subject_id = s.id) AS file_count
  FROM subjects s
 WHERE s.is_archived = 0;
```

10.7.4 Derived table (subquery in FROM)

```
SELECT a.id, a.name, t.recent_actions
  FROM (
            SELECT account_id, COUNT(*) AS recent_actions
            FROM activity_log
            WHERE performed_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
            GROUP BY account_id
  ) t
  JOIN accounts a ON a.id = t.account_id
 ORDER BY t.recent_actions DESC
 LIMIT 5;
```

Dashboard
My Mentees
Analytics
Live Feed

Monitor performance , track activity, and review mentee progress.

Academic Highlights

TOP PERFORMERS		MENTOR WORKLOAD	
1 Om Mishra	2064 MINUTES	Prof. Rajesh Kumar	51 ACTIVE
2 Tanvi Walia	2040 MINUTES	Aarav Mentor	47 ACTIVE
3 Zara Khan	2000 MINUTES	Dr. Priya Sharma	44 ACTIVE
4 Aditya Kumar	1571 MINUTES	Anita Desai	44 ACTIVE
5 Pari Sehgal	1545 MINUTES	Sandeep Gupta	42 ACTIVE
6 Krish Choudhary	1501 MINUTES	Dr. Ramya Krishnan	35 ACTIVE
7 Meera Pillai	1324 MINUTES	Dr. Vikram Singh	33 ACTIVE
8 Manav Shah	1173	Devraj Mentor	32

Recent Updates

VIEW ALL ACTIVITY

- Tarun Nayak add assessment session — 7h ago
- Nikhil Roy notification read subject — 7h ago
- Amit Bhatia task created subject — 7h ago
- Meera Pillai login task — 9h ago
- Madhav Rao started a study session study session — 12h ago
- Rajat Reddy started a study session study session — 12h ago
- Rajat Reddy started a study session study session — 12h ago

Engagement

ROSTER HEALTH

100%

Percentage of your mentees who have logged activity in the last 48 hours.

Quick Links

Manage Students

10.8 Functions

10.8.1 Aggregate functions

```
SELECT de.habit_id,
        COUNT(*)          AS score_rows,
        SUM(sc.score) AS total_score,
        AVG(sc.score) AS average_score,
        MIN(sc.score) AS lowest_score,
        MAX(sc.score) AS highest_score
 FROM dailyentries de
 JOIN scores sc ON sc.entry_id = de.entry_id
GROUP BY de.habit_id;
```

10.8.2 Built-in scalar functions

```
SELECT vf.id,
        UPPER(vf.file_type)                    AS type_upper,
        LOWER(vf.file_name)                    AS name_lower,
        LENGTH(vf.note_description)            AS desc_length,
        COALESCE(vf.note_name, vf.file_name,'?') AS display_name,
        NULLIF(vf.file_size_kb, 0)             AS non_zero_size,
        ROUND(vf.file_size_kb / 1024, 2)       AS size_mb,
        DATE_FORMAT(vf.uploaded_at, '%Y-%m-%d')  AS uploaded_date,
        DAYNAME(vf.uploaded_at)                AS uploaded_weekday,
        EXTRACT(YEAR FROM vf.uploaded_at)          AS uploaded_year,
        NOW()                            AS query_time
 FROM vault_files vf;
```

10.8.3 User-defined scalar functions

upgrades.sql defines four:

- fn_account_storage_kb(account_id) — total KB across all files.
- fn_habit_average_score(habit_id) — average score across all entries.
- fn_subject_total_items(subject_id) — files + links count.
- fn_user_consistency_streak(user_id) — consecutive-day streak using a CURSOR + LOOP + control flow.

```
CREATE FUNCTION fn_user_consistency_streak(p_user_id INT)
RETURNS INT
READS SQL DATA
BEGIN
        DECLARE v_streak        INT  DEFAULT 0;
        DECLARE v_prev_date DATE DEFAULT NULL;
        DECLARE v_cur_date  DATE;
        DECLARE v_done          INT  DEFAULT 0;
        DECLARE cur CURSOR FOR
        SELECT DISTINCT entry_date FROM dailyentries
        WHERE user_id = p_user_id ORDER BY entry_date DESC;
        DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

        OPEN cur;
        streak_loop: LOOP
        FETCH cur INTO v_cur_date;
        IF v_done = 1 THEN LEAVE streak_loop; END IF;

        IF v_prev_date IS NULL THEN
        IF v_cur_date >= DATE_SUB(CURDATE(), INTERVAL 1 DAY) THEN
        SET v_streak = 1; SET v_prev_date = v_cur_date;
        ELSE LEAVE streak_loop; END IF;
        ELSEIF DATEDIFF(v_prev_date, v_cur_date) = 1 THEN
        SET v_streak = v_streak + 1; SET v_prev_date = v_cur_date;
        ELSE LEAVE streak_loop; END IF;
        END LOOP;
        CLOSE cur;
        RETURN v_streak;
END
```

Used inline like a built-in:

```
SELECT u.user_id, u.username,
        fn_user_consistency_streak(u.user_id) AS streak_days,
        fn_account_storage_kb(a.id)           AS account_kb
 FROM users u
 LEFT JOIN accounts a ON a.email = u.email;
```

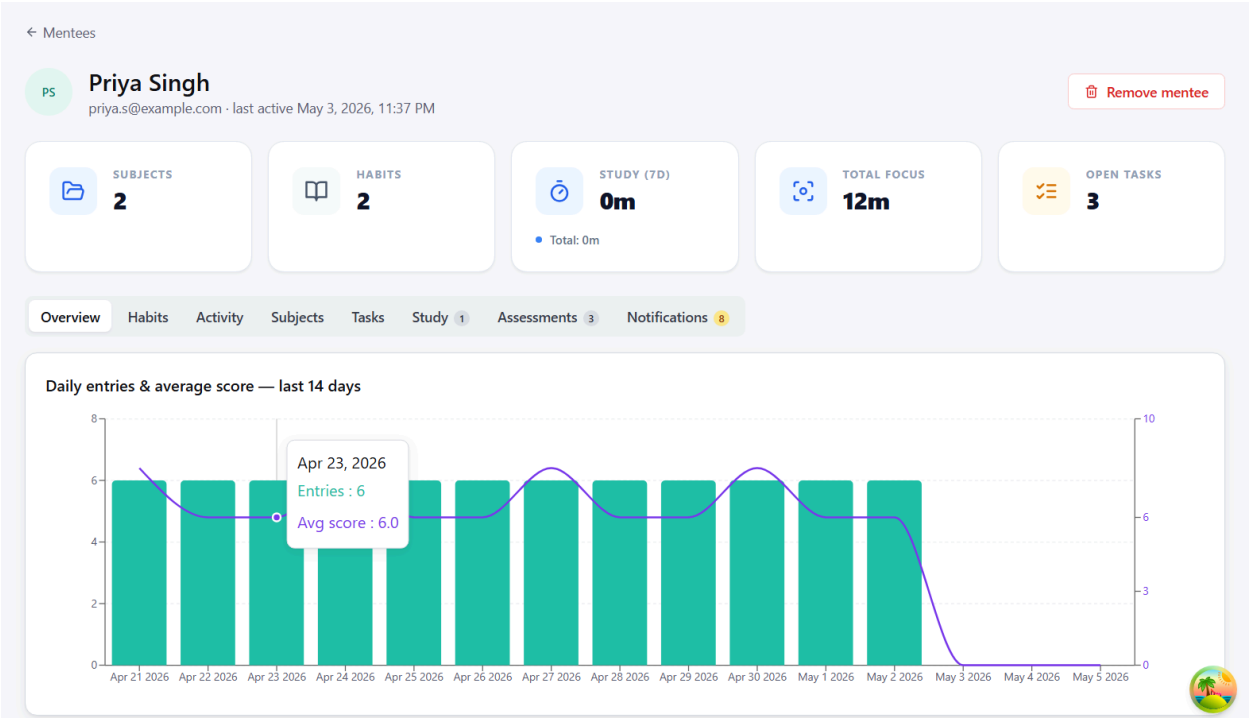
Explanation: User-defined functions such as `fn_user_consistency_streak` are essential for StudyLab's habit analytics feature. This function calculates consecutive active days, which is used to measure student discipline and display streak-based insights in the application.

Output:

Full Performance Summary

SQL VIEW + AGGREGATION

STUDENT NAME	SUBJECTS	TOTAL STUDY (MINS)	COMPLETED TASKS
Priya Singh	2	0.3	0 / 3
Rohan Joshi	2	0	0 / 2
Neha Kapoor	2	0	0 / 2



10.9 Views

upgrades.sql and mentor_crm.sql define eight views that act as the project's reporting layer:

View	Purpose
vw_account_public	Privacy-safe projection over accounts (hides password_hash).
vw_account_overview	Per-account vault roll-up: subjects / files / links + total storage.
vw_subject_overview	Per-subject roll-up consumed by GET /api/subjects.
vw_habit_score_stats	Per-habit COUNT / MIN / MAX / AVG / STDDEV across entries × questions.
vw_daily_score_heatmap	One row per (user, day) for the calendar / heatmap UI.
vw_top_engaged_accounts	Top accounts by combined vault engagement (HAVING demo).
vw_question_difficulty	CASE-WHEN bands (easy / moderate / hard) per question.
vw_mentor_roster	A mentor's roster with subjects / files / habits / 7-day-entries / open-tasks.

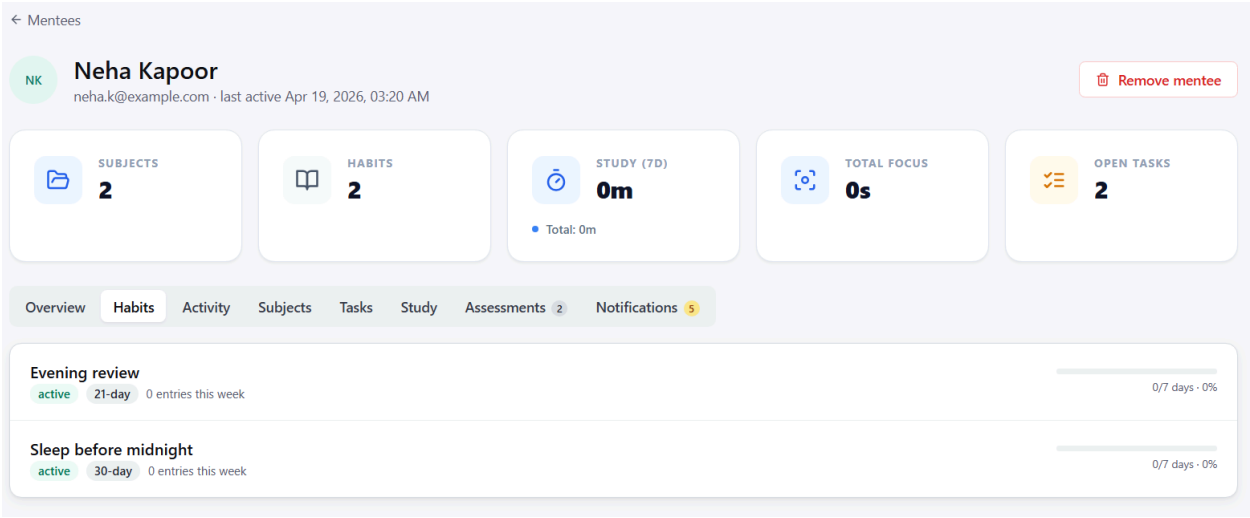
Example: vw_account_overview showcases LEFT JOIN, aggregates, scalar functions, NULLIF, and GREATEST in one definition:

```
CREATE OR REPLACE VIEW vw_account_overview AS
SELECT
        a.id                    AS account_id,
        a.name                          AS account_name,
        UPPER(a.email)                  AS email_upper,
        a.role,
        a.is_active,
        COUNT(DISTINCT s.id)                    AS subject_count,
        COUNT(DISTINCT vf.id)                   AS file_count,
        COUNT(DISTINCT vl.id)                   AS link_count,
        COALESCE(SUM(vf.file_size_kb), 0) AS total_storage_kb,
        ROUND(COALESCE(SUM(vf.file_size_kb), 0) / NULLIF(1024, 0), 2)
                                AS total_storage_mb,
        GREATEST(
        COALESCE(MAX(vf.uploaded_at), '1970-01-01'),
        COALESCE(MAX(vl.added_at),        '1970-01-01'),
        COALESCE(MAX(s.created_at),   '1970-01-01')
        )                       AS last_activity_at
FROM accounts a
LEFT JOIN subjects              s  ON s.firebase_uid = a.id AND s.is_archived = 0
LEFT JOIN vault_files vf ON vf.subject_id  = s.id
LEFT JOIN vault_links vl ON vl.subject_id  = s.id
GROUP BY a.id, a.name, a.email, a.role, a.is_active;
```

Explanation:

Views such as `vw_mentor_roster` and `vw_account_overview` directly support frontend APIs in StudyLab. Instead of writing complex joins repeatedly in backend code, the application queries these views to fetch aggregated data like student engagement and subject statistics efficiently.

Output:



10.10 Stored Procedures

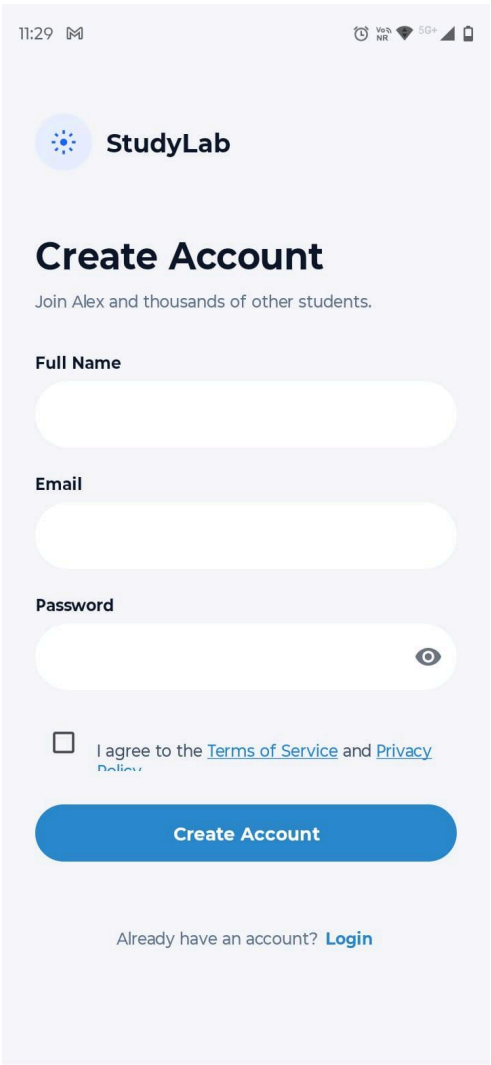
Procedure	Demonstrates
proc_register_account	Atomic 3-write (account + welcome notification + audit) with START TRANSACTION / COMMIT / handler ROLLBACK / SIGNAL.
proc_archive_subject	Update inside a transaction with ownership guard.
proc_log_daily_entry	INSERT/UPDATE + JSON loop + SAVEPOINT + score-range validation.
proc_recompute_habit_analysis	DELETE + INSERT-from-SELECT with aggregates inside a transaction.
proc_mark_notifications_read	Bulk UPDATE + conditional audit insert.

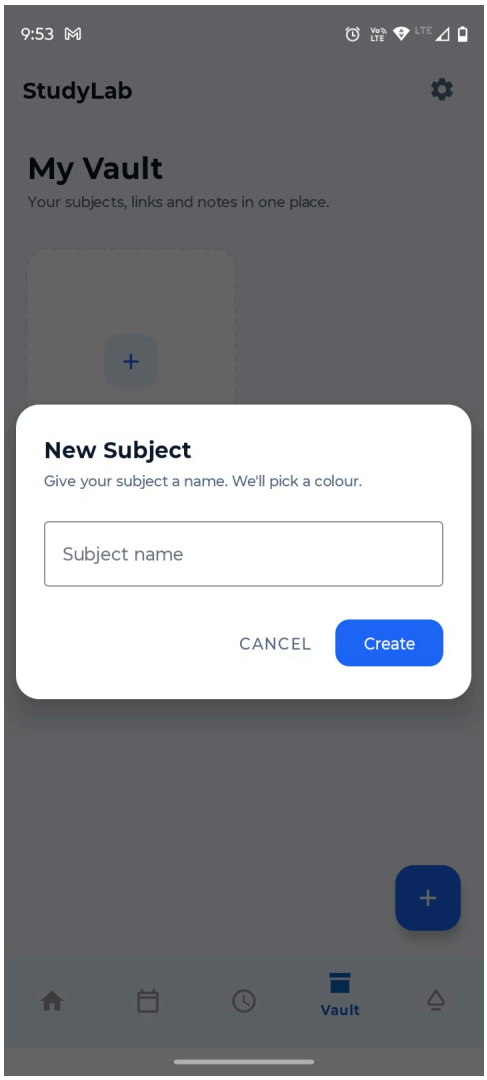
proc_register_account is the simplest illustration:

```
CREATE PROCEDURE proc_register_account(
        IN p_id             VARCHAR(36),
        IN p_name           VARCHAR(100),
        IN p_email          VARCHAR(255),
        IN p_password_hash VARCHAR(255),
        IN p_role           VARCHAR(20)
)
BEGIN
        DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN
        ROLLBACK;
        RESIGNAL;
        END;

        IF (SELECT COUNT(*) FROM accounts WHERE email = p_email) > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Email already registered';
        END IF;

        START TRANSACTION;

        INSERT INTO accounts (id, name, email, password_hash, role)
        VALUES (p_id, p_name, p_email, p_password_hash,
        COALESCE(p_role, 'student'));

        INSERT INTO notifications (id, account_id, type, message, link)
        VALUES (UUID(), p_id, 'WELCOME',
        CONCAT('Welcome, ', p_name,
        '! Create your first subject to begin.'),
        '/vault');

        INSERT INTO activity_log (account_id, action_type, entity_type,
                entity_id, metadata)
        VALUES (p_id, 'REGISTER', 'account', p_id,
        JSON_OBJECT('email', p_email,
```

```
                    'role',  COALESCE(p_role, 'student')));

            COMMIT;
END
```

Output:





10.11 Triggers

upgrades.sql, mentor_crm.sql and assessments.sql together define nine AFTER triggers that maintain audit + notification fan-out automatically:

Trigger	Fires on	Effect
trg_accounts_after_insert	INSERT accounts	Append REGISTER row to activity_log.

trg_accounts_after_update	UPDATE accounts	Audit only meaningful changes (email / role / is_active).
trg_subjects_after_insert	INSERT subjects	Audit row with new metadata.
trg_subjects_after_delete	DELETE subjects	Audit row with old metadata.
trg_vault_files_after_insert	INSERT vault_files	Audit row keyed on the parent subject's owner.
trg_vault_links_after_insert	INSERT vault_links	Same pattern.
trg_dailyentries_after_insert	INSERT daily entries	Append to activity_log.
trg_habits_status_change	UPDATE habits to 'completed'	Fan out a celebratory notification.
trg_mentor_tasks_after_insert	INSERT mentor_tasks	Notification to student + audit row to mentor.
trg_assessments_after_insert	INSERT assessments	Notification to student + audit row to mentor (or self).

Example — the assessment trigger:

```
CREATE TRIGGER trg_assessments_after_insert
AFTER INSERT ON assessments
FOR EACH ROW
BEGIN
        INSERT INTO notifications (id, account_id, type, message, link)
        VALUES (UUID(), NEW.student_id, 'NEW_ASSESSMENT',
        CONCAT('New ', NEW.type, ': ', NEW.title),
        CONCAT('/student/assessments/', NEW.id));

        INSERT INTO activity_log (account_id, action_type, entity_type,
                entity_id, metadata)
        VALUES (COALESCE(NEW.mentor_id, NEW.student_id), 'CREATE_ASSESSMENT',
        'assessment', NEW.id,
        JSON_OBJECT('student_id', NEW.student_id,
                'title',    NEW.title,
                'type',     NEW.type,
                'due_at',   NEW.due_at,
                'self',     NEW.mentor_id IS NULL));
END
```

Explanation:

Triggers play a key role in StudyLab by automatically maintaining the activity log and notification system. For instance, when a mentor assigns a task, the trigger ensures that the student instantly receives a notification without requiring additional application logic. This guarantees consistency across both Android and web clients.

Output

Overview | Habits | **Activity** | Subjects | Tasks | Study 1 | Assessments 3 | Notifications 8

completed a daily check-in check in	1d ago
student update task mentor task	1d ago
wrapped up a study session study session	1d ago
focus on focus event	1d ago
started a study session study session	1d ago
insert vault link	2d ago
insert vault link	2d ago

Overview | Habits | Activity | Subjects | Tasks | Study | Assessments 2 | **Notifications 5**

new assessment UNREAD
New project: Re-engage project: pick a topic
May 3, 2026, 11:00 PM

new assessment UNREAD
New assignment: Self-study: Strang chapter 4
May 3, 2026, 11:00 PM

new task UNREAD
New task from your mentor: DO YOUR HOMEWORK
May 3, 2026, 10:24 PM

new task UNREAD
New task from your mentor: Set up daily review habit
May 3, 2026, 03:20 AM

streak missed UNREAD
You missed your check-in yesterday.
May 2, 2026, 03:20 AM

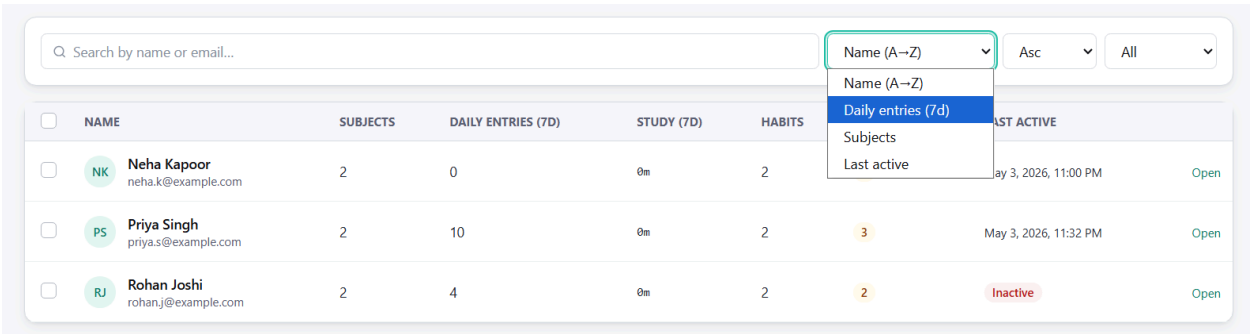
11. Output Explanation

11.1 Mentor's roster — view-backed

Query:

```
SELECT student_id, student_name, student_email,
            subjects_count, files_count, habits_count,
            daily_entries_7d, open_tasks, last_login_at
 FROM vw_mentor_roster
 WHERE mentor_id = ?
 ORDER BY daily_entries_7d DESC, student_name;
```

Sample output:

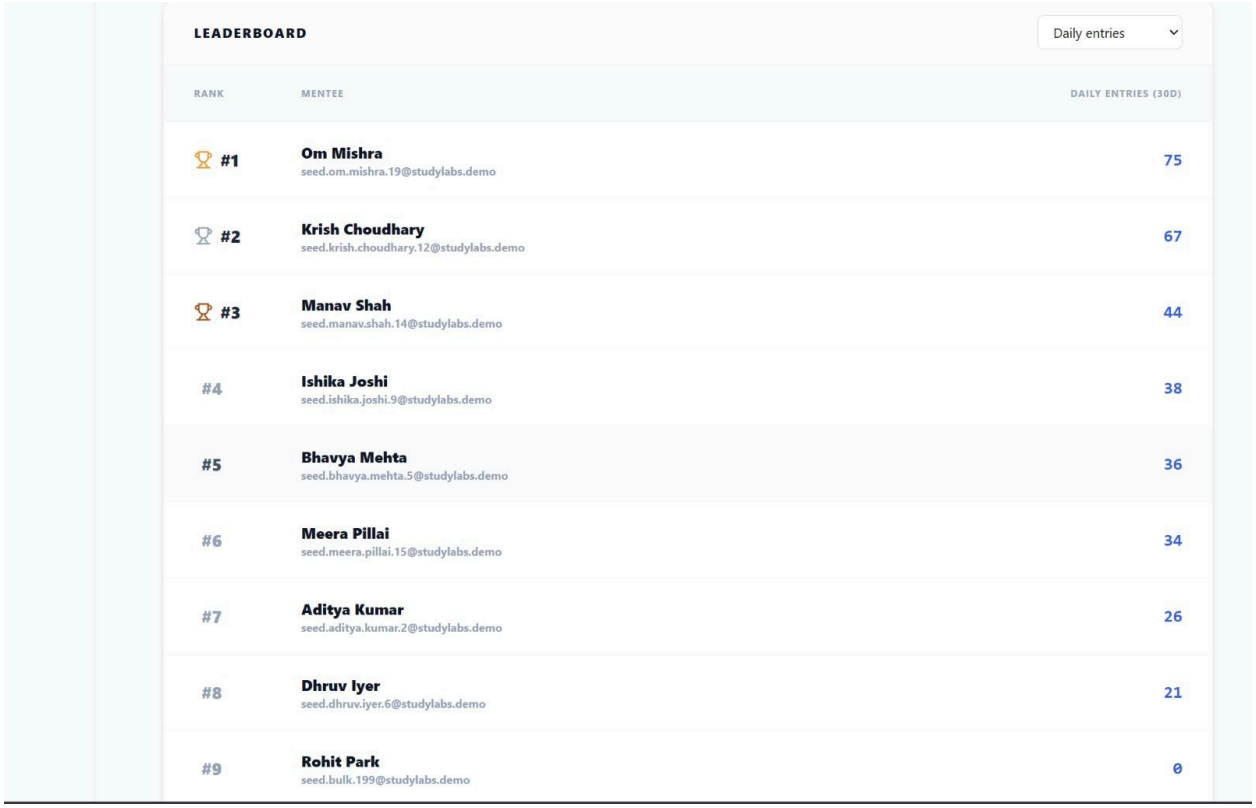


Why & what it means: vw_mentor_roster hides the four-way LEFT JOIN across mentor_assignments → accounts → users → habits / dailyentries / subjects / vault_files / mentor_tasks — the route issues a single SELECT * FROM vw_mentor_roster WHERE mentor_id = ?. Sorting by daily_entries_7d DESC puts the most-active mentees on top; mentees with 0 (Neha) are flagged as "at-risk" in the UI.

11.2 Top-engaged accounts (HAVING)

```
SELECT * FROM vw_top_engaged_accounts ORDER BY engagement_score DESC LIMIT 10;
```

Sample output:



LEADERBOARD

Daily entries

RANK	MENTEE	DAILY ENTRIES (30D)
#1	Om Mishra seed.om.mishra.19@studylabs.demo	75
#2	Krish Choudhary seed.krish.choudhary.12@studylabs.demo	67
#3	Manav Shah seed.manav.shah.14@studylabs.demo	44
#4	Ishika Joshi seed.ishika.joshi.9@studylabs.demo	38
#5	Bhavya Mehta seed.bhavya.mehta.5@studylabs.demo	36
#6	Meera Pillai seed.meera.pillai.15@studylabs.demo	34
#7	Aditya Kumar seed.aditya.kumar.2@studylabs.demo	26
#8	Dhruv Iyer seed.dhruv.iyer.6@studylabs.demo	21
#9	Rohit Park seed.bulk.199@studylabs.demo	0

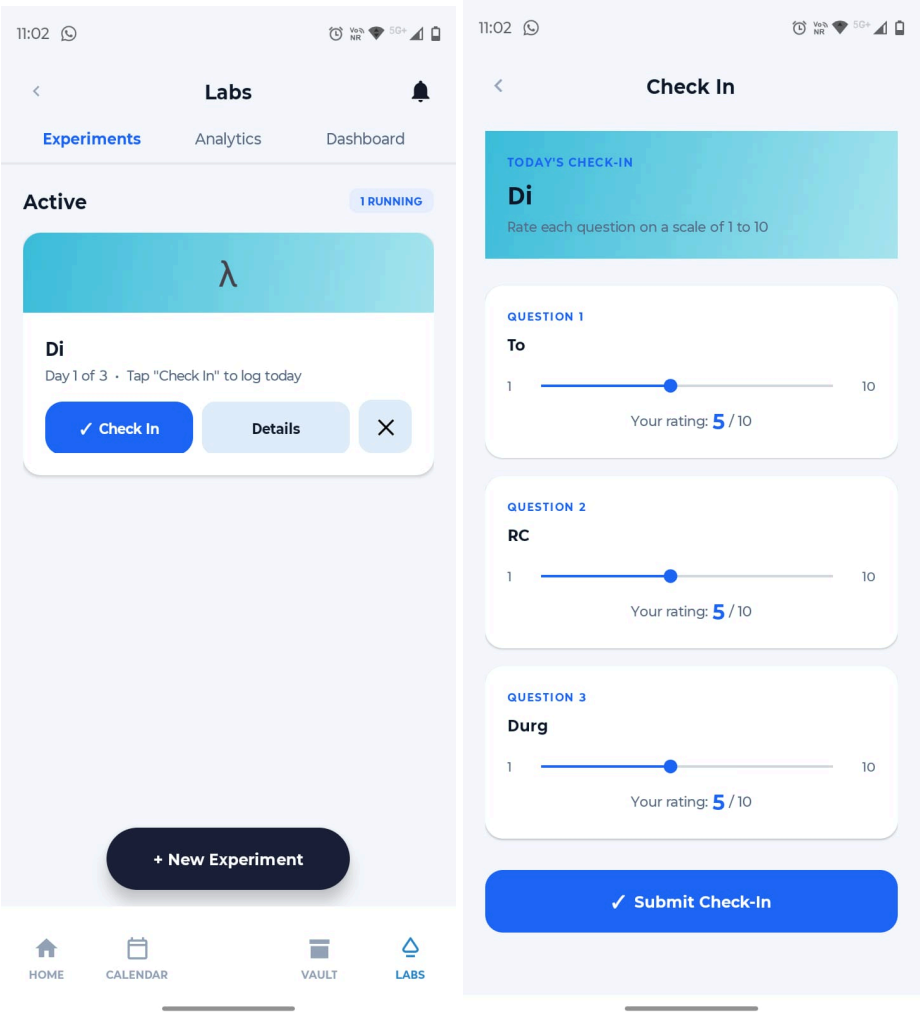
Explanation:

The view aggregates three counts and returns only rows whose sum is non-zero, expressed as HAVING engagement_score > 0. WHERE cannot reference an aggregate alias — that's exactly the textbook reason HAVING exists. Drives the mentor leaderboard on /mentor/analytics.

11.3 Habit average via user-defined scalar function

```
SELECT h.habit_id,
       h.habit_name,
       fn_habit_average_score(h.habit_id) AS avg_score
 FROM habits h
 WHERE h.user_id = ?
 ORDER BY avg_score DESC;
```

habit_id	habit_name	avg_score
14	Deep work block	8.40
11	Sleep before 11 pm	7.10
17	Mood check-in	5.85



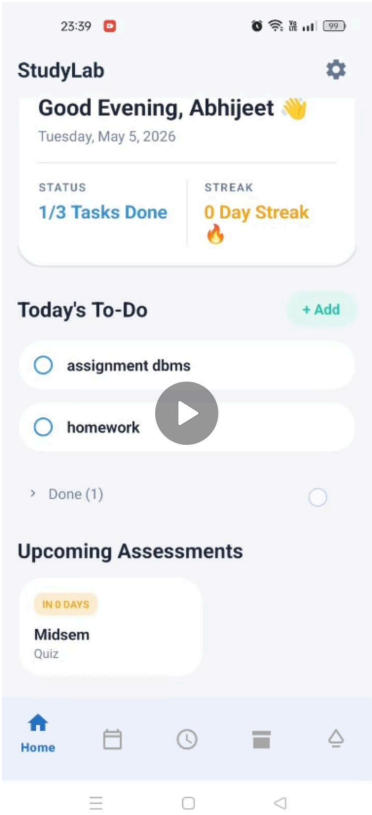
Explanation:

fn_habit_average_score encapsulates the JOIN+AVG so the same logic can be reused in views, procedures, and ad-hoc queries. Returns NULL if the habit has no scores yet — the UI shows "No data yet" instead of 0.00.

11.4 Atomic daily check-in (procedure with SAVEPOINT)

Call:

```
CALL proc_log_daily_entry(
        1,                              -- p_user_id
        1,                              -- p_habit_id
        CURDATE(),                              -- p_entry_date
        'Felt focused this morning',            -- p_daily_note
  '[{"question_id":1,"score":8},{"question_id":2,"score":7}]',
        @new_entry_id
);
SELECT @new_entry_id;
```



Output (happy path):

```
@new_entry_id = 4093
```

activity_log shows: log_id=8821, account_id=NULL, action_type='INSERT', entity_type='dailyentry', entity_id='4093' — courtesy of trg_dailyentries_after_insert.

Output (bad payload — score = 11):

```
ERROR 1644 (45000): Score must be in [0..10]
```

Explanation:

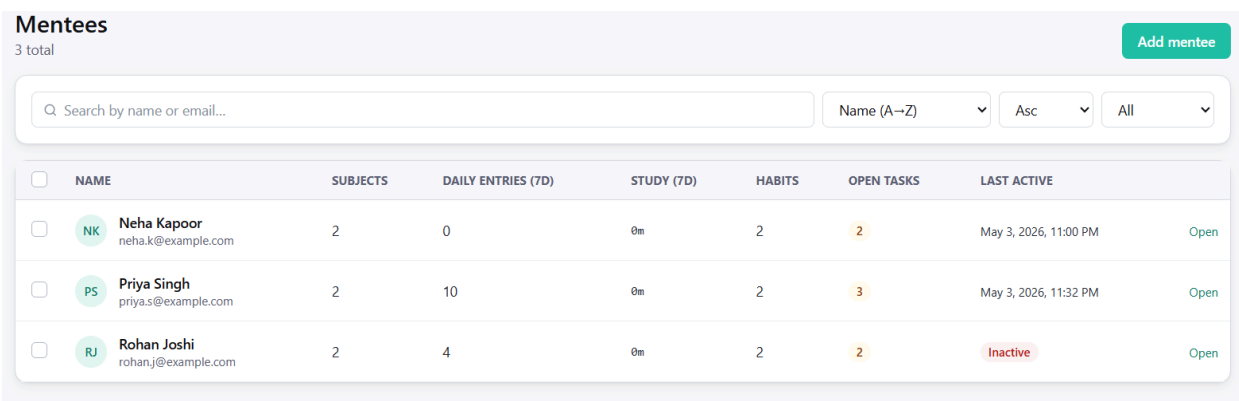
The procedure inserts the dailyentries row, marks SAVEPOINT before_scores, then loops over the JSON. If any score is out of range, ROLLBACK TO SAVEPOINT before_scores discards only the score inserts — the daily entry survives and the mentor still sees that the student attempted to log today.

11.5 Bulk task assignment with trigger fan-out

Backend code path (POST /api/mentor/tasks/bulk) — inside withTransaction():

```
INSERT INTO mentor_tasks (id, mentor_id, student_id, title,
                          description, due_date, priority)
SELECT UUID(), :mentor_id, s.id, :title, :description,
            :due_date, :priority
  FROM accounts s
 WHERE s.id IN (:student_ids)
   AND EXISTS (
            SELECT 1 FROM mentor_assignments
            WHERE mentor_account_id  = :mentor_id
            AND student_account_id = s.id
   );
```

Output:



Sample output — viewing the result:

```
SELECT n.account_id, n.message, n.link, n.created_at
  FROM notifications n
 WHERE n.account_id IN (:student_ids)
   AND n.type = 'NEW_TASK'
 ORDER BY n.created_at DESC;
```

12. Conclusion

StudyLab demonstrates the practical application of DBMS concepts in building a real-world academic system. By integrating student and mentor functionalities into a single MySQL database, the project ensures consistency, scalability, and efficient data handling across multiple domains.

The system successfully addresses challenges such as fragmented study resources, lack of habit tracking, and limited mentor visibility. Through normalization, indexing, views, and transactions, it maintains data integrity while supporting complex queries and analytics.

In conclusion, StudyLab goes beyond a theoretical database project by presenting a functional and scalable solution that improves academic organization, accountability, and performance through effective database design.

ROLES OF THE MEMBERS

In our project, each member has been assigned specific roles and responsibilities:

1. Aakriti : worked on complete development of vault feature, report, frontend.
2. Aryamanan Pratap Singh : worked on API integration, Login and authorization and report
3. Abhijeet Gupta: worked on mentor feature development, study focus and video
4. Arnav Gupta: worked on complete development of lab experiment feature, graphs and ppt

AI Use

- We have utilized Ai for helping in the ER diagram designing and ideation.
- As well as for writing advanced and complex sql queries that were needed.
- We also took assistance from Ai for identifying on how to apply rubrics on our application.